

4.1 PROGRAMMING

Programming is talking to computer. It is a language to communicate with your computer. The main unit of a computer is microprocessor. Each microprocessor is designed to execute a certain set of instructions. Through programming, we pass these instructions to computer and computer executes these instructions in given order.

A Computer language is defined as a code or a syntax which is used to write programs or any specific applications. Computer language is used to communicate with computers. Broadly, it can be classified into three categories assembly language, machine language, and high-level language. The machine language is considered as oldest computer language among all three. In machine language, the input is directly given as binary input which is processed by the machine. Binary inputs mean one and zero form. For computer language processing the system needs compiler and interpreter to convert the language in computer language so that it can be processed by a machine.

What is inside a Microprocessor?

A microprocessor contains millions of transistors and capacitive elements. All these components operate after providing voltages. Presence of voltage means a signal 1 and absence of voltage means signal 0. So, a computer is only able to understand presence and absence of voltages.

Machine Language to C

Computer only understands machine language also called binary language. A machine language instruction consists of only 0s and 1s. For example, to add two numbers, machine language instruction can be 00011001. It is very difficult to remember and understand these machine language instructions for programmers.

To make the instructions understandable to the programmer, assembly language was formed. Assembly language consists of some simple statements i.e., sum a, b. This instruction is supposed to add a and b. Each processor has its own assembly language instruction set. So, it can only execute instructions from its instruction set. Whenever an assembly language instruction is executed, it is first converted into equivalent machine

language instruction (consisting of 0s and 1s).

Although assembly language is easier to understand for the programmers, there is still missing information in assembly language instructions, for example, *sum a, b* is equivalent to $a = a + b$, but it cannot be inferred from the instruction that result will be assigned to *a*. To facilitate the programmers further, another language was designed named “C Language”. This language was implemented to write programs in mathematical language. In C language, if we have to sum *a* and *b* and store the result in *a*, the instruction will be: $a = a + b$. There is no hidden information in this statement.

What is Programming?

When a C program is executed, it is first translated into equivalent assembly language code, which is further translated into equivalent machine language code. The computer finally executes machine language instructions being translated from C program. Hence a C program is “a **set of sequence of commands** being sent to microprocessor for execution.”

Programming is not only writing instructions to the computer but it is instructions in the right sequence to form a required logic (i.e., to sum *a* and *b* etc.).

Integrated Development Environment for C

An IDE, or Integrated Development Environment, enables programmers to consolidate the different aspects of writing a computer program. IDEs increase programmer productivity by combining common activities of writing software into a single application: editing source code, building executables, and debugging.

Compilation of a C program converts C code to machine language instructions. There are many softwares available for compiling and running C programs. Some famous IDEs are Microsoft Visual Studio, Visual C++ and Eclipse etc. Online compilers can also be explored. In this chapter, we will only learn to use Microsoft Visual Studio.

Do you know?

There is a step wise debugger available in many IDEs which helps you to troubleshoot your program.

Point to Ponder

- What is the difference between C and C++ languages?

4.2 Different Types of Computer Languages:

Below are the top 3 types of computer language:

1. Machine Language

The machine language is sometimes referred to as machine code or object code which is a set of binary digits: 0 and 1. These binary digits are understood and read by a computer system and it interprets it easily. It is considered a native language as it can be directly understood by a central processing unit (CPU). The machine language is not so easy to understand, as the language uses the binary system in which the commands are written in 1 and 0 form which is not easy to interpret. There is only one language which is understood by computer language which is machine language. The operating system of the computer system is used to identify the exact machine language used for that particular system.

The operating system defines how the program should be written so that it can be converted to machine language and the system takes appropriate action. The computer programs and scripts can also be written in other programming languages like C, C++, and JAVA. However, these languages cannot be directly understood by a computer system so there is a need for a program that can convert these computer programs to machine language. The compiler is used to convert the programs to machine language which can be easily understood by computer systems. The compiler generates the binary file and executable file.

Example of machine language for the text “Hello World”.

01001000 0110101 01101100 01101100 01101111 00100000 01010111 01101111
01110010 01101100 01100100.

2. Assembly Language

The assembly language is considered a low-level language for microprocessors and many other programmable devices. The assembly language is also considered as second-generation language. The first-generation language is machine language. The assembly language is mostly famous for writing an operating system and also in writing different desktop applications. The operations carried out by programmers using assembly language are memory management, registry access, and clock cycle operations. The drawback of assembly language is that the code cannot be reused and the language is not so easy to understand. The assembly language is considered a group of other languages. It is used to implement the symbolic representation of machine code which is used to program CPU architecture. The other name of assembly language is assembly code. For any processor, the most used programming language is assembly language.

In assembly language, the programmer does the operation which can be directly executed on a central processing unit (CPU). The language has certain drawbacks as it does not contain any variables or functions in programs and also the program is not portable on different processors. The assembly language uses the same structure and commands which machine language does use but it uses names in place of numbers. The operations performed using the assembly language is very fast. The operations are much faster when it is compared to high-level language.

Activity

- Find *sum* command for different microcontrollers from their datasheets.

Do you know

- Every microprocessor has its own assembly language instructions.

3. High-Level Languages

The code written in low-level language for one machine cannot be transferred to other machines. Thus, it led to the development of high-level language. The high-level language is easy to understand as the programs written are user-friendly. The other advantage of code written in a high-level language is that the code is independent of a computer system which means the code can be transferred to other machines. The high-level of language uses the concept of abstraction and also focus on programming language rather than focusing on computer hardware components like register utilization or memory utilization.

Higher-level languages were created to write a human-readable program that can be easily understood by any user. The only requirement in a high-level language is the need of compiler. The program written in a high-level language is not directly understood by the computer. It needs to be converted to machine level language before the execution of high-level programs. The examples of high-level language are C++, C, JAVA, FORTRAN and Python etc.

When you are programming to control the behavior of hardware components, then low level programming languages are used. For Instance, C is used to interface sensors with microcontrollers. Whereas, Python is used for Artificial Intelligence.

4.3 Language Translators

A translator is a programming language processor that converts a computer program from one language to another. It takes a program written in source code and converts it into machine code. It discovers and identifies the error during translation.

Purpose of Translator

It translates high-level language program into a machine language program that the central processing unit (CPU) can understand. It also detects errors in the program.

Roles of translator are

- Translating the high-level language program input into an equivalent machine language program.
- Providing diagnostic messages wherever the programmer violates specification of the high-level language program.

Different Types of Translators

There are 3 different types of translators as follows:

Compiler

A compiler is a translator used to convert high-level programming language to low-level programming language. It converts the whole program in one session and reports errors detected after the conversion. Compiler takes time to do its work as it translates high-level code to lower-level code all at once and then saves it to memory.

A compiler is processor-dependent and platform-dependent. This dependence has been addressed by a special compiler, a cross-compiler (which compiles a program for another different microprocessor) and a source-to-source compiler. Before choosing a compiler, user has to identify first the Instruction Set Architecture (ISA), the operating system (OS) and the programming language that will be used to ensure that it will be compatible.

Interpreter

Just like a compiler, interpreter is a translator used to convert high-level programming language to low-level programming language. It converts the program one at a time and reports errors detected at once, while doing the conversion. With this, it is easier to detect errors than in a compiler. An interpreter is faster than a compiler as it immediately executes the code upon reading the code.

It is often used as a debugging tool for software development as it can execute a single line of code at a time. An interpreter is also more portable than a compiler as it is not processor-dependent, you can work between hardware architectures.

Assembler

An assembler is a translator used to translate assembly language to machine language. It is like a compiler for the assembly language but interactive like an interpreter. Assembly language is difficult to understand as it is a low-level programming language. An assembler translates a low-level language, an assembly language to an even lower-level language, which is the machine code. The machine code can be directly understood by the CPU.

4.4 Program Development Life Cycle

When we develop a program using any programming language, we follow a sequence of steps. These steps are called phases in program development. The program development life cycle is a set of steps or phases that are used to develop a program in any programming language.

Generally, the program development life cycle contains 6 phases as shown in Figure 4.1, they are as follows:

- Problem Definition
- Problem Analysis
- Algorithm Development
- Coding & Documentation
- Testing & Debugging
- Maintenance

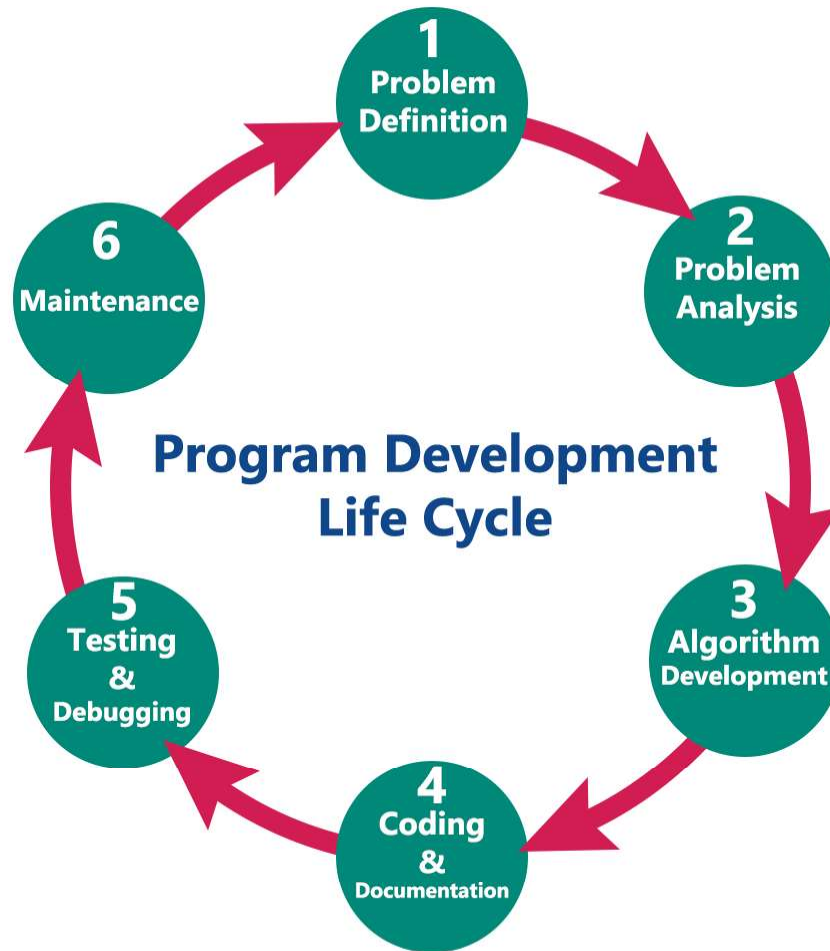


Fig. 4.1 Program Development Life Cycle

1. Problem Definition

In this phase, we define the problem statement and we decide the boundaries of the problem. In this phase we need to understand the problem statement, what is our requirement, what should be the output of the problem solution. These are defined in this first phase of the program development life cycle.

2. Problem Analysis

In phase 2, we determine the requirements like variables, functions, etc. to solve the problem. That means we gather the required resources to solve the problem defined in the problem definition phase. We also determine the bounds of the solution.

3. Algorithm Development

During this phase, we develop a step-by-step procedure to solve the problem using the specification given in the previous phase. This phase is very important for program development.

4. Coding & Documentation

This phase uses a programming language to write or implement the actual programming instructions for the steps defined in the previous phase. In this phase, we construct the actual program. That means we write the program to solve the given problem using programming languages like C, C++, Java, etc.,

5. Testing & Debugging

During this phase, we check whether the code written in the previous step is solving the specified problem or not. We also test whether it is providing the desired output or not.

6. Maintenance

During this phase, the program is actively used by the users. If the user encounters any problem or wants any enhancement, then we need to repeat all the phases from the starting, so that the encountered problem is solved or enhancement is added.

4.5 Flow Charts

A flowchart is simply a graphical representation of steps. It shows steps in sequential order and is widely used in presenting the flow of algorithms, workflow or processes. Typically, a flowchart shows the order of steps by connecting them with arrows. Figure 4.2 shows an example flowchart.

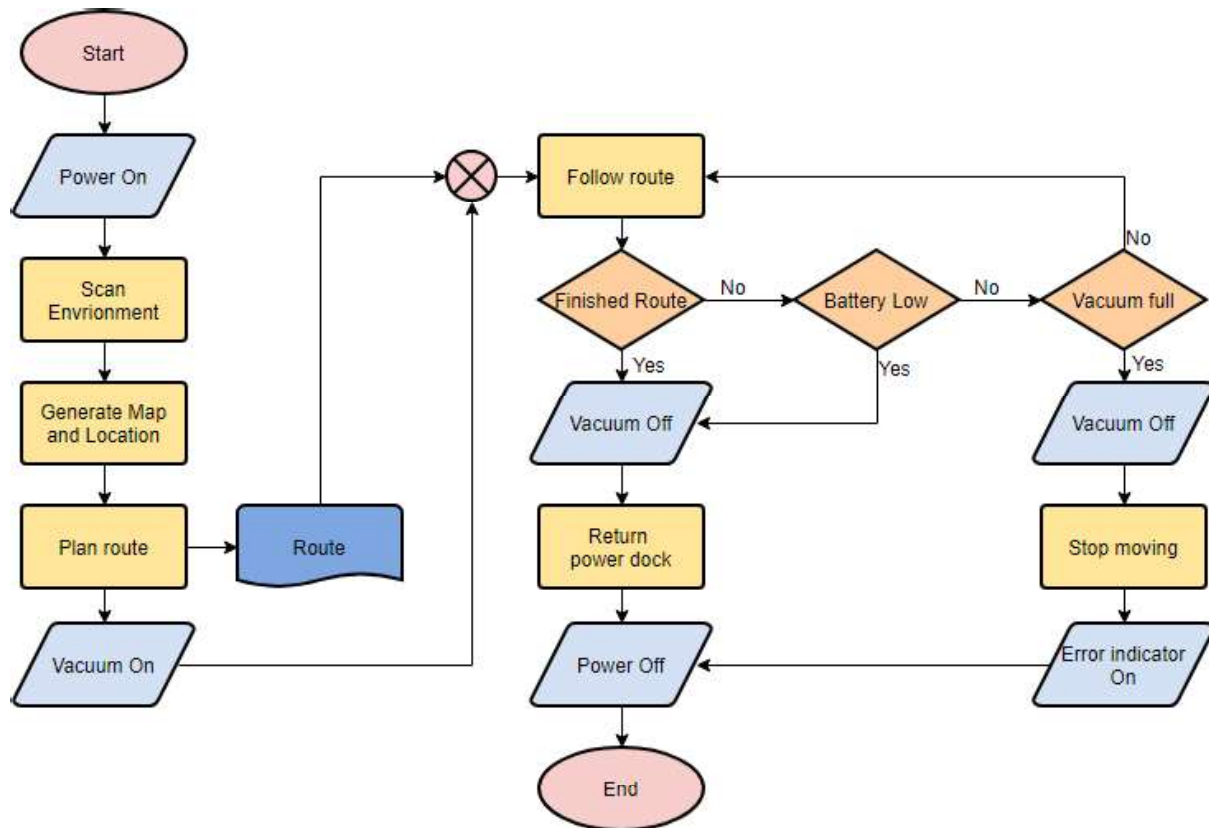


Fig. 4.2 Flow Chart Example

A flowchart is a graphical representation of steps. It was originated from computer science as a tool for representing algorithms and programming logic but had extended to use in all other kinds of processes. Nowadays, flowcharts play an extremely important role in displaying information and assisting reasoning. They help us visualize complex processes, or make explicit the structure of problems and tasks. A flowchart can also be used to define a process or project to be implemented.

4.5.1 Flow chart Symbols

Different flowchart shapes have different conventional meanings. The meanings of some of the more common shapes are as follows:

Terminator

The terminator symbol represents the starting or ending point of the system.



Process

A box indicates some particular operation.



Document

This represents a printout, such as a document or a report.



Decision

A diamond represents a decision or branching point. Lines coming out from the diamond indicates different possible situations, leading to different sub-processes.



Data

It represents information entering or leaving the system. An input might be an order from a customer. Output can be a product to be delivered.



On-Page Reference

This symbol would contain a letter inside. It indicates that the flow continues on a matching symbol containing the same letter somewhere else on the same page.



Off-Page Reference

This symbol would contain a letter inside. It indicates that the flow continues on a matching symbol containing the same letter somewhere else on a different page.



Delay

Identifies a delay.



Flow

Lines represent the flow of the sequence and direction of a process.



Benefits of Flowcharts

Using a flowchart has a variety of benefits:

- It helps to clarify complex processes.
- It identifies steps that do not add value to the internal or external customer, including delays; needless storage and transportation; unnecessary work, duplication, and added expense; breakdowns in communication.
- It helps team members gain a shared understanding of the process and use this knowledge to collect data, identify problems, focus discussions, and identify resources.
- It serves as a basis for designing new processes.

4.5.2 Flowchart Examples

In this section, we present some flowchart examples.

Medical Service

This is a hospital flowchart example that shows how clinical cases shall be processed. This flowchart (Figure 4.3) uses decision shapes intensively in representing alternative flows.

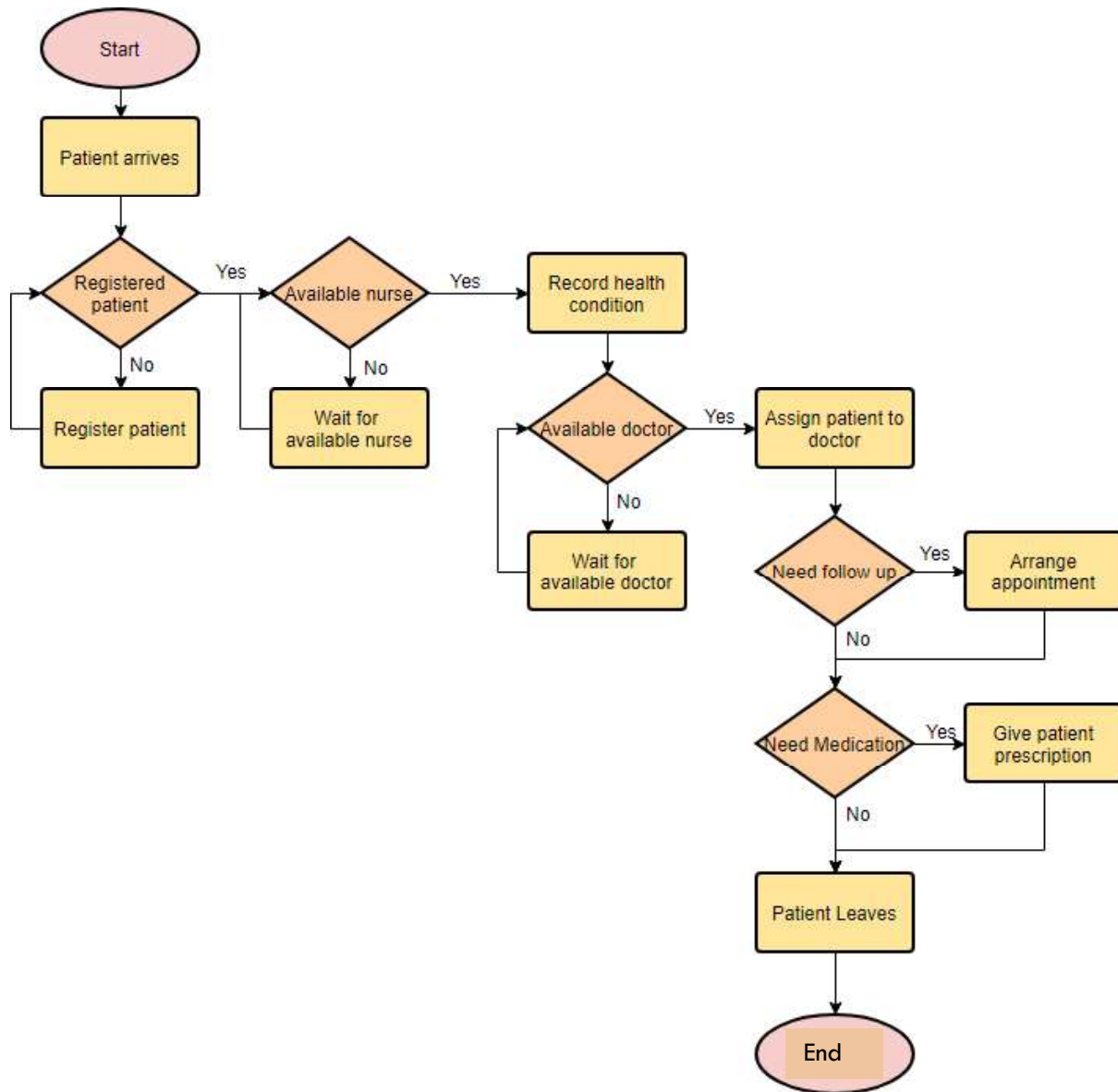


Fig. 4.3 Flow Chart Example: Medical

Simple Algorithms

A flowchart can also be used in visualizing algorithms, regardless of its complexity. Here is an example (Figure 4.4) that shows how flowchart can be used in showing a simple summation process.

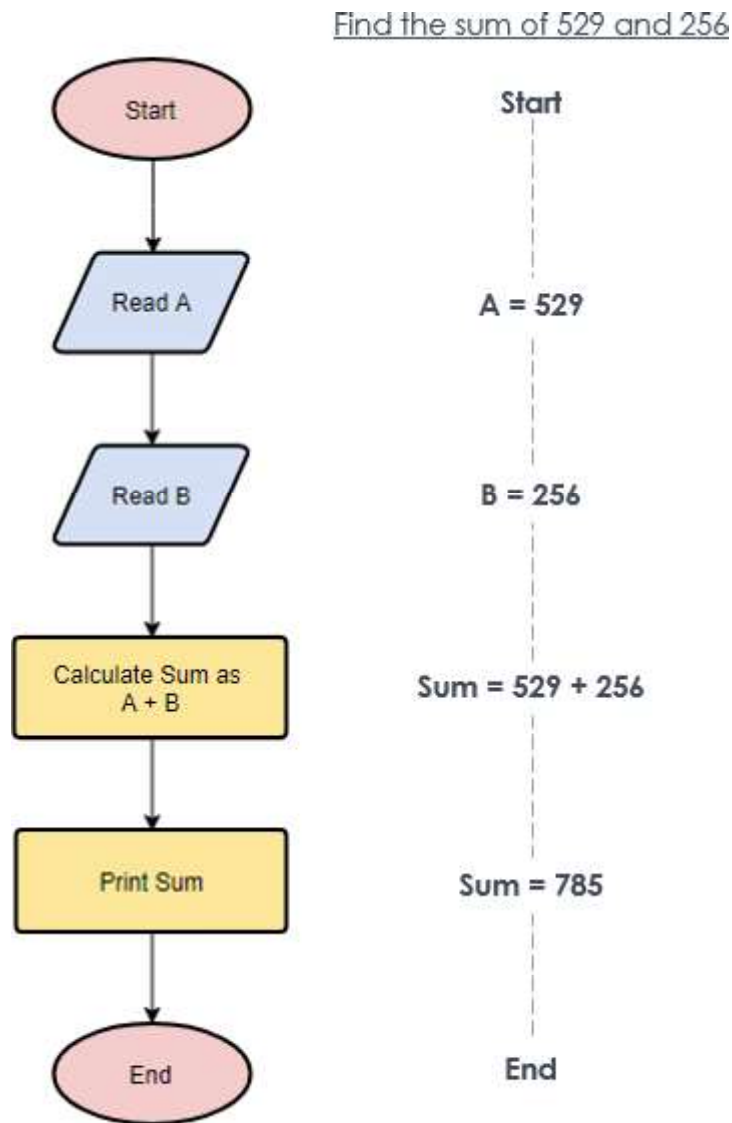


Fig. 4.4 Flow Chart Example: Sum

Calculate Profit and Loss

The flowchart example below (Figure 4.5) shows how profit and loss can be calculated.

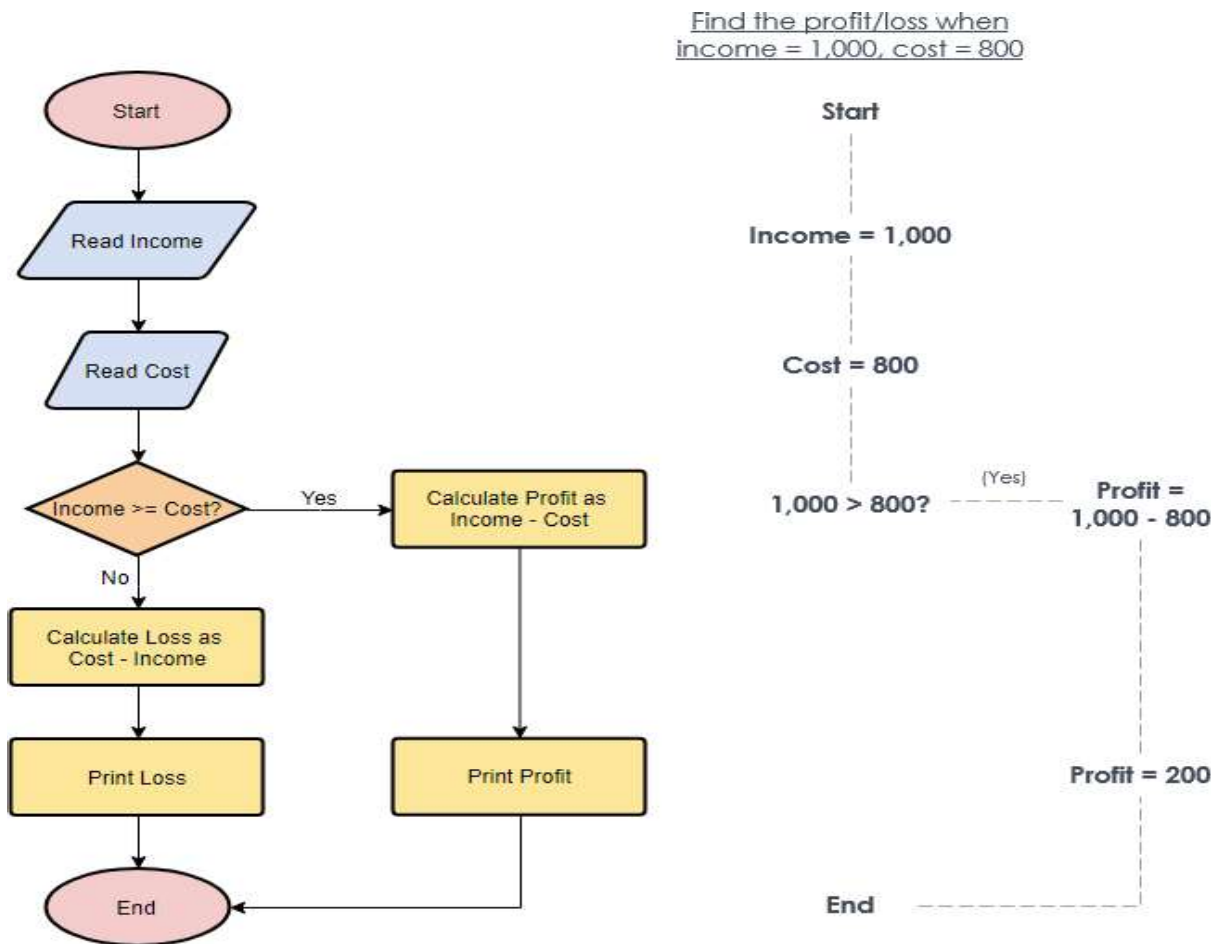


Fig. 4.4 Flow Chart Example: Profit and Loss Calculation

Note for Teachers.

- Give examples of recursive flow charts.

Key Points

- The computer language is defined as code or syntax which is used to write programs or any specific applications.
- The main unit of a computer is microprocessor. Each microprocessor is designed to execute a certain set of instructions.
- Presence of voltage means a signal 1 and absence of voltage means signal 0. So, a computer is only able to understand presence and absence of voltages.
- Computer only understands machine language also called binary language.
- Assemble language consists of some simple statements i.e., sum a, b.
- C language was implemented to write programs in mathematical language.
- Hence a C program is “a **set of sequence of commands** being sent to microprocessor for execution.”
- An IDE, or Integrated Development Environment, enables programmers to consolidate the different aspects of writing a computer program.
- There is a step wise debugger available in many IDEs which helps you to troubleshoot your program.
- The other advantage of code written in a high-level language is the code is independent of a computer system which means the code can be transferred to other machines.
- A translator is a programming language processor that converts a computer program from one language to another.
- An assembler is a translator used to translate assembly language to machine language

Exercise

Select the most appropriate option

1. Assembly is a
 - a. High level language
 - b. Low level language
 - c. Machine Language
 - d. Both a and b
2. Which of the following language is different for all microcontrollers.
 - a. Assembly Language
 - b. Machine Language
 - c. C Language
 - d. Both a and b
3. _____ converts the assembly language code to machine language.
 - a. Compiler
 - b. Assembler
 - c. Linker
 - d. None of these
4. Which of the following language supports mathematical expressions.
 - a. C
 - b. Assembly
 - c. Machine Language
 - d. none of these
5. _____ are used to understand flow of a computer algorithm.
 - a. Flow charts
 - b. Line graphs
 - c. Bar charts
 - d. none of these

write short answer of the following.

1. What is programming?
2. What is the difference between high level and low level programming languages?
3. Define program development cycle.
4. What is the difference between a debugger and a compiler.

Answer the following question in detail.

1. Explain program translation steps?
2. What are flow chart and why are they used?
3. Differentiate between Interpreter and Compiler.